Decoding Personalities: A Deep Dive into MBTI Data Analysis

1 Overview of the project's purpose and objectives

1.1 Introduction

Understanding how people behave, and think is crucial, especially in such a diverse society like Singapore. In this data analysis, we explore the intricacies of the 16 personality types among Singaporeans, using the Myers-Briggs Type Indicator (MBTI) as a framework to analyze the complex interplay of traits. This personality test is a self-reporting questionnaire that was developed by Cook Briggs and her daughter, Isabel Briggs Myers based on the theories presented by Carl Jung.

Like people worldwide, Singaporeans exhibit a wide range of personality traits which adds richness to our society. The MBTI test assesses personalities based on four specific differences which further categorizes them into 16 personality types.

This essay aims to highlight the diverse expressions of personality within the Singaporean population by examining the four different types of indicators and the 16 personality types. The use of four features will help to produce the MBTI results.

- **Energy:** This feature shows how we interact with our surroundings.
  - **E:** Energized by people, enjoy a variety of tasks, a quick pace, and are good at multitasking.
  - **I:** Prefer working alone or in small groups, favor a more deliberate pace, and like to focus on one task at a time.
- **Mind:** This second detail determines how we see the world and process information.
  - **S:** Realistic and like to focus on facts and details and apply common sense and experience to come up with practical solutions to problems.
  - **N:** Prefer to focus on possibilities and the big picture, easily see patterns, value innovation, and seek creative solutions to problems.
- **Nature:** The third detail determines how we make decisions and cope with emotions. Its personality types: Thinking (T) vs. Feeling (F).
  - **T:** Make decisions using logical analysis, objectively weigh pros and cons, and value honesty, consistency, and fairness.
  - **F:** Tend to be sensitive and cooperative and decide based on their values and how others will be affected by their actions.
- **Tactics:** The final characteristic reflects our approach to work, planning, and decision-making.
  - **J:** Tend to be organized and prepared, like to make and stick to plans, and are comfortable following more rules.
  - **P:** Prefer to keep their options open, like to be able to act spontaneously, and enjoy being flexible with making plans.

Based on the findings and MBTI results, individuals can be categorized into the four role groups:

- **Analysts (xNTx):** Including personality types such as INTJ, INTP, ENTJ, ENTP.
- **Diplomats (xNFx):** Encompassing personality types like INFJ, INFP, ENFJ, ENFP.
- **Sentinels (xSxJ):** Consisting of personality types such as ISTJ, ISFJ, ESTJ, ESFJ.
- **Explorers (xSxP):** Enlisting personality types like ISTP, ISFP, ESTP, ESFP.

With the help of data science methods, we can explore the subtleties of MBTI personality types among Singaporeans in this study. Despite the Myers-Briggs Type Indicator (MBTI) not being widely recognized in Singapore, it may still offer a valuable framework for understanding personality dynamics.

1.2 Aim & Objective

For this research and data collection from Singaporeans, we use the 16 Personalities test website for its internal consistency, test-retest reliability, and discriminant validity. Furthermore, this website is frequently used in Singaporean schools to guide students in their career exploration. Students are encouraged to take the MBTI to better understand their strengths and preferences, which helps them make informed decisions on their future careers. By analyzing large datasets related to personality traits, we can uncover patterns, correlations, and unique characteristics associated with each personality type.

Within this project, we aim to explore the following areas:

- Determine the most common MBTI types, traits, roles, and strategies among individuals in Singapore and providing an overall analysis of common personality preferences within the population.
- Conduct a comparative analysis of MBTI distributions between Singapore and other regions.
- Perform an analysis of different MBTI types within the Singaporean population.
- Examine the distribution of specific personality traits and their importance in shaping individual behaviors and attitudes.

This report also provides an outlook on the distribution of MBTI types in Singapore to identify the most and least popular personality preferences. At the same time, offering insights into the dominant and less common personality profiles within the population. Despite the relative awareness in Singapore, the goal of this report is to provide a deeper understanding of personality dynamics within Singaporeans based on available data.

le data.

1.3 Data

1.3.1 Data requirement

For this study, 16Personalities test website (https://www.16personalities.com/free-personality- test) was exclusively chosen to gather information about Singaporeans' personality types. This decision was made to ensure consistency and reliability in the data. The 16Personalities test is known for its internal consistency, test-retest reliability, and discriminant validity. Using multiple MBTI websites could introduce variations and inconsistencies which will not produce close to accurate results.

Additionally, this website is frequently used in Singaporean schools to guide students in their ca reer exploration. Students are encouraged to take the MBTI to better understand their strengths and preferences, which helps them make informed decisions about their future careers. The test consists of 130 questions that take approximately 10-15 minutes to complete. Students are given options based on how they feel, ranging from "strongly agree" to "strongly disagree."The preferences are then submitted and used to identify their results.

The 16 personality types are identified by an abbreviation of the first letters of each of the four type preferences for each cognitive function: extraversion/introversion, sensing/intuition, thinking/feeling, and judging/perceiving.

For example, “ISTP” stands for introversion, sensing, thinking, and perceiving. No combination is considered “better” or “worse” than another—all types are considered equal.

Another source that will be used for the analysis is the online forum platform Reddit. Many Singaporeans use Reddit to discuss various topics. By examining Reddit, I can gain insights into the personalities of Singaporean users. This involves understanding their interests, emotions, social connections, and popular discussions. Analyzing these details will allow to better understand MBTI trends within the Singaporean community. Data from Reddit discussions will be cross -referenced with the 16Personalities results to validate the findings and identify any discrepancies. This willhelp ensure the accuracy of ults.

1.3.2 Choice of data

The 16 Personalities website is valuable because it offers resources for examining personality profiles based on regions and countries. Through the Singapore personality profile page (https://www.16personalities.com/country-profiles/singapore), we can gather results specifically from Singaporean respondents. In this dataset, we will:

- Gather the abbreviations representing the first letters of each of the four personality type traits: I or E, S or N, F or T, J or P.
- Collect data on the different role groups: Analysts (xNTx), Diplomats (xNFx), Sentinels (xSxJ), and Explorers (xSxP).
- According to the webpage, out of Singapore's estimated population of 5,674,472, a total of 653,413 respondents have been included.

For Reddit, we will utilize their API service, PRAW, to access posts, comments, votes, and other information from various subreddits (r/) and users (u/). To use the API, a client ID and a secret token have been registered on the Reddit app.

1.3.4 Limitations and constraints of the data

MBTI and its results may not always be dependable.

- **Sampling Bias:** The reliability of MBTI results can be affected if certain groups, like people with multiple accounts or those really interested in personality tests, are overrepresented in the data. This can make the results not truly reflect the whole population.

- **Incomplete Data:** If we don't have access to enough different sources of data about MBTI, the results might not tell the whole story. For instance, if there's not much data available in Singapore about MBTI as it is not a popular topic here, our analysis might be missing important information, which could make it less reliable. Specific information that might be missing includes age, gender, location, and career.

- **Test-Retest Reliability:** Sometimes, when people take the MBTI test again and again, they can get different results each time. This makes it hard to trust the test's consistency over time.

- **Self-Selection Bias:** People who take the MBTI test do so by choice. This means the results might be skewed towards those already interested in personality types.

- **Self-Reporting Issues:** The MBTI relies on how people report their traits. Factors like mood, honesty, and understanding of the questions can affect the accuracy of these reports.

- **Cultural Bias:** The MBTI was designed within a specific culture. It might not fully capture personality traits across different cultures, which can limit its applicability to Singaporeans.

- **Limited Representation:** Not everyone can easily access or complete the MBTI test. This includes individuals without internet access or those who don't speak the language well, which may lead to an unrepresentative sample.

1.4 Ethical considerations

1.4.1 Compliance with Terms

To ensure compliance with the terms and conditions of the 16Personalities website, the following steps were taken:

- **Permission to Use Data:** The 16Personalities team was contacted via email to request permission to use data from their website for research purposes. Consent was granted on the condition that proper citations are included.
- **Data Usage:** Data collected from the 16Personalities website is used strictly for personal research purposes, adhering to the restrictions outlined in their terms of use. This includes:
  - Viewing and downloading content for caching purposes only.
  - Ensuring no commercial exploitation or public communication of the data.
  - Including proper citations for all data used to acknowledge the source.

By adhering to these guidelines, the use of data from the 16Personalities website is compliant with their terms and conditions and ensures ethical and legal use of the provided resources.

In [1]:
```
pip install wordcloud
```

```
Requirement already satisfied: wordcloud in c:\users\jolin\anaconda3\lib\site-packages (1.9.3)
Requirement already satisfied: numpy>=1.6.1 in c:\users\jolin\anaconda3\lib\site-packages (from wordcloud) (1.26
.4)
Requirement already satisfied: pillow in c:\users\jolin\anaconda3\lib\site-packages (from wordcloud) (10.2.0)
Requirement already satisfied: matplotlib in c:\users\jolin\anaconda3\lib\site-packages (from wordcloud) (3.8.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\jolin\anaconda3\lib\site-packages (from matplotlib->
wordcloud) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\jolin\anaconda3\lib\site-packages (from matplotlib->word
cloud) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\jolin\anaconda3\lib\site-packages (from matplotlib-
>wordcloud) (4.25.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\jolin\anaconda3\lib\site-packages (from matplotlib-
>wordcloud) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\jolin\anaconda3\lib\site-packages (from matplotlib->w
ordcloud) (23.1)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\jolin\anaconda3\lib\site-packages (from matplotlib->
wordcloud) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\jolin\anaconda3\lib\site-packages (from matplotl
ib->wordcloud) (2.8.2)
Requirement already satisfied: six>=1.5 in c:\users\jolin\anaconda3\lib\site-packages (from python-dateutil>=2.7
->matplotlib->wordcloud) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
pip install nltk
```

```
Requirement already satisfied: nltk in c:\users\jolin\anaconda3\lib\site-packages (3.8.1)
Requirement already satisfied: click in c:\users\jolin\anaconda3\lib\site-packages (from nltk) (8.1.7)
Requirement already satisfied: joblib in c:\users\jolin\anaconda3\lib\site-packages (from nltk) (1.2.0)
Requirement already satisfied: regex>=2021.8.3 in c:\users\jolin\anaconda3\lib\site-packages (from nltk) (2023.1
0.3)
Requirement already satisfied: tqdm in c:\users\jolin\anaconda3\lib\site-packages (from nltk) (4.65.0)
Requirement already satisfied: colorama in c:\users\jolin\anaconda3\lib\site-packages (from click->nltk) (0.4.6)
Note: you may need to restart the kernel to use updated packages.
```

In [3]:
```
# Import libraries and modules
import pandas as pd
from bs4 import BeautifulSoup
import requests
import json
from collections import defaultdict
import praw
import re
import pandas as pd
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.sentiment import SentimentIntensityAnalyzer
from nltk.corpus import stopwords
from wordcloud import WordCloud
from collections import Counter


import matplotlib.pyplot as plt
```

2 Web Scraping for MBTI Personality trait Analysis

2.1 Web Page Accessibility and Data Extraction

Before proceeding with the web scraping project, it's crucial to ensure that the target webpages are accessible. In this section, we describe the function developed to verify the accessibility of webpages and retrieve their content if accessible. We identify the webpage URL from which we want to retrieve data. Having the correct URL to access the desired resource effectively is crucial. In our case, we are interested in retrieving data from the country profile page of Singapore on the 16Personalities website at "https://www.16personalities.com/country-profiles/singapore"

In [38]:
```
def top10webpage(url):
    # Send a GET request
    response = requests.get(url)

    # Check if the request was successful
    if response.status_code == 200:
        # Parse HTML content
        soup = BeautifulSoup(response.content, "html.parser")
        return soup
    else:
        print("Failed to retrieve data. Status code:", response.status_code)
        return None

url = "https://www.16personalities.com/country-profiles/singapore"
soup = top10webpage(url)
```

2.1.1 Extracting information from webpage

Since we successfully accessed the webpage, we're ready to extract specific information. Below is a function tailored to retrieve data concerning the top personality types from the Singapore personality webpage:

In [39]:
```
def top10personalitySG(soup):
    # Initialize a string to store the content
    content = ""

    # Find the specific header
    header = soup.find("h3", class_="title")
    if header and "Top personality types" in header.text:
        content += header.text.strip() + "\n"

        # Find the country-profiles-top-ten-list component
        component = soup.find("country-profiles-top-ten-list")
        if component and ":collection" in component.attrs:
            content += "Data found.\n"
        else:
            content += "Component not found or collection attribute missing.\n"
    else:
        content += "Header not found or text does not match.\n"

    return content

# Call the function
url = "https://www.16personalities.com/country-profiles/singapore"
soup = top10webpage(url)
```

This function systematically searches for a specific header containing the text "Top personality types" within the webpage's content. If found, it proceeds to extract information regarding the top personality types. Additional code for extracting specific information would be added within the function as needed, depending on the structure of the webpage and the desired data.

2.1.2 Import URLs and scrape

Moving forward, we'll import the CSV files containing global information sourced from the official website's global country personality profiles. This data will be instrumental in our subsequent analysis and reporting.

As we've already retrieved the data from the website and stored it in a CSV file, we can read it into a pandas DataFrame.

In [40]:
```
# Read the global.csv file into a DataFrame
global_df = pd.read_csv('globaltraits.csv')

# Read the globalroles.csv file into a DataFrame
global_roles_df = pd.read_csv('globalroles.csv')

# Print the first few rows of each DataFrame to verify the data
print("globaltraits.csv:")
print(global_df.head())

print("\nglobalroles.csv:")
print(global_roles_df.head())
```

```
globaltraits.csv:
  trait  Percentage
0     E        50.0
1     I        50.0
2     S        45.0
3     N        55.0
4     F        54.5

globalroles.csv:
  Role Group  Percentage
0         NT        21.2
1         NF        40.7
2         SJ        23.0
3         SP        15.1
```

2.2 Data cleaning and processing¶

2.2.1 Standardizing data format

2.2.1.1 Using JSON for Extraction, Cleaning, and Parsing of JSON-like Strings

The data we require is embedded within the HTML structure of the webpage, specifically within the :collection attribute of the element. This JSON-like string contains valuable information about the personality types that we want to analyze further. However, the data retrieved from the webpage is in a JSON-like format, and it contains HTML encoding such as " instead of standard double quotes ("). Without extracting this string, we cannot access the raw data needed for analysis. To properly interpret this data as JSON and work with it in Python, we need to ensure that the string is formatted correctly. Therefore, we extract the JSON-like string and convert it to a proper JSON string. Once the JSON-like string is cleaned and formatted correctly, we parse it into a Python dictionary to manipulate and analyze the data.

In [41]:
```
# Define the component variable
component = soup.find("country-profiles-top-ten-list")

# Extract the JSON-like string
json_data = component.attrs[":collection"]

# Convert the JSON-like string to a proper JSON string
json_data = json_data.replace('&quot;', '"')

# Parse the JSON string
data = json.loads(json_data)
print(data)
```

```
[{'code': 'infp', 'name': 'Turbulent Mediator', 'percentage': '12.55'}, {'code': 'enfp', 'name': 'Turbulent Camp
aigner', 'percentage': '8.40'}, {'code': 'isfj', 'name': 'Turbulent Defender', 'percentage': '7.56'}, {'code': '
isfp', 'name': 'Turbulent Adventurer', 'percentage': '7.43'}, {'code': 'esfj', 'name': 'Assertive Consul', 'perc
entage': '5.37'}, {'code': 'infj', 'name': 'Turbulent Advocate', 'percentage': '5.06'}, {'code': 'esfp', 'name':
'Turbulent Entertainer', 'percentage': '4.81'}, {'code': 'enfp', 'name': 'Assertive Campaigner', 'percentage': '
4.20'}, {'code': 'esfj', 'name': 'Turbulent Consul', 'percentage': '4.07'}, {'code': 'isfj', 'name': 'Assertive
Defender', 'percentage': '3.99'}, {'code': 'esfp', 'name': 'Assertive Entertainer', 'percentage': '3.87'}, {'cod
e': 'infp', 'name': 'Assertive Mediator', 'percentage': '3.01'}, {'code': 'isfp', 'name': 'Assertive Adventurer'
, 'percentage': '2.98'}, {'code': 'enfj', 'name': 'Turbulent Protagonist', 'percentage': '2.57'}, {'code': 'intp
', 'name': 'Turbulent Logician', 'percentage': '2.47'}, {'code': 'enfj', 'name': 'Assertive Protagonist', 'perce
ntage': '1.95'}, {'code': 'estj', 'name': 'Assertive Executive', 'percentage': '1.88'}, {'code': 'istj', 'name':
'Assertive Logistician', 'percentage': '1.66'}, {'code': 'istj', 'name': 'Turbulent Logistician', 'percentage':
'1.55'}, {'code': 'infj', 'name': 'Assertive Advocate', 'percentage': '1.48'}, {'code': 'intp', 'name': 'Asserti
ve Logician', 'percentage': '1.47'}, {'code': 'entp', 'name': 'Turbulent Debater', 'percentage': '1.45'}, {'code
': 'istp', 'name': 'Turbulent Virtuoso', 'percentage': '1.37'}, {'code': 'entp', 'name': 'Assertive Debater', 'p
ercentage': '1.34'}, {'code': 'estp', 'name': 'Assertive Entrepreneur', 'percentage': '1.23'}, {'code': 'intj',
'name': 'Turbulent Architect', 'percentage': '1.11'}, {'code': 'istp', 'name': 'Assertive Virtuoso', 'percentage
': '1.06'}, {'code': 'intj', 'name': 'Assertive Architect', 'percentage': '0.98'}, {'code': 'estj', 'name': 'Tur
bulent Executive', 'percentage': '0.93'}, {'code': 'estp', 'name': 'Turbulent Entrepreneur', 'percentage': '0.93
'}, {'code': 'entj', 'name': 'Assertive Commander', 'percentage': '0.73'}, {'code': 'entj', 'name': 'Turbulent C
ommander', 'percentage': '0.53'}]
```

Now, the raw data from the webpage has been converted into a structured format that we can effectively work with in Python.

2.2.2 Remove duplicates and aggregating dataset

After retrieving the data, it is observed that some personalities are repeated twice within the dataset. This duplication arises from the separation of personalities into Assertive and Turbulent categories, as seen with examples like Mediator-A and Mediator-T. We intend to merge them with their corresponding percentages for this analysis. We start by creating a dictionary to hold the aggregated percentages for each personality type. This dictionary will iterate through each item in the data list, which contains the personality data. For each item, we extract the name of the personality and its associated percentage. The name is then cleaned by removing the substrings "Assertive " and "Turbulent". After cleaning, the name and percentage are added to the dictionary. If the personality already exists in the dictionary, its percentage is incremented by the new percentage value. This process ensures that each personality type is represented once in the aggregated data, eliminating any duplicates caused by the presence of "Assertive" and "Turbulent" labels. The aggregated data are then sorted in descending order based on their percentage.

In [8]:
```python
# Aggregate the percentages by name
aggregated_data = defaultdict(float)
for item in data:
    name = item['name'].replace("Assertive ", "").replace("Turbulent ", "")
    percentage = float(item['percentage'])
    aggregated_data[name] += percentage

# Sort the aggregated data by percentage in descending order
sorted_data = sorted(aggregated_data.items(), key=lambda x: x[1], reverse=True)

# Print the aggregated and sorted data with ranking
for rank, (name, percentage) in enumerate(sorted_data, start=1):
    print(f"Rank {rank}: Name: {name}, Percentage: {percentage:.2f}%")
```

```
Rank 1: Name: Mediator, Percentage: 15.56%
Rank 2: Name: Campaigner, Percentage: 12.60%
Rank 3: Name: Defender, Percentage: 11.55%
Rank 4: Name: Adventurer, Percentage: 10.41%
Rank 5: Name: Consul, Percentage: 9.44%
Rank 6: Name: Entertainer, Percentage: 8.68%
Rank 7: Name: Advocate, Percentage: 6.54%
Rank 8: Name: Protagonist, Percentage: 4.52%
Rank 9: Name: Logician, Percentage: 3.94%
Rank 10: Name: Logistician, Percentage: 3.21%
Rank 11: Name: Executive, Percentage: 2.81%
Rank 12: Name: Debater, Percentage: 2.79%
Rank 13: Name: Virtuoso, Percentage: 2.43%
Rank 14: Name: Entrepreneur, Percentage: 2.16%
Rank 15: Name: Architect, Percentage: 2.09%
Rank 16: Name: Commander, Percentage: 1.26%
```

We utilize a dictionary named 'name_to_code' to map each personality name to its corresponding code. This dictionary contains key-value pairs where the personality names ('Mediator', 'Campaigner', etc.) are mapped to their respective codes ('INFP', 'ENFP', etc.)

In [9]:
```python
# Dictionary to map names to codes
name_to_code = {
    'Mediator': 'INFP',
    'Campaigner': 'ENFP',
    'Defender': 'ISFJ',
    'Adventurer': 'ISFP',
    'Consul': 'ESFJ',
    'Entertainer': 'ESFP',
    'Advocate': 'INFJ',
    'Protagonist': 'ENFJ',
    'Logician': 'INTP',
    'Logistician': 'ISTJ',
    'Executive': 'ESTJ',
    'Debater': 'ENTP',
    'Virtuoso': 'ISTP',
    'Entrepreneur': 'ESTP',
    'Architect': 'INTJ',
    'Commander': 'ENTJ'
}
# Create a DataFrame from the sorted data
df = pd.DataFrame(sorted_data, columns=['Name', 'Percentage'])

# Update the Name column using the dictionary
df['Code'] = df['Name'].map(name_to_code)
df
```

Out[9]:

	Name	Percentage	Code
0	Mediator	15.56	INFP
1	Campaigner	12.60	ENFP
2	Defender	11.55	ISFJ
3	Adventurer	10.41	ISFP
4	Consul	9.44	ESFJ
5	Entertainer	8.68	ESFP
6	Advocate	6.54	INFJ
7	Protagonist	4.52	ENFJ
8	Logician	3.94	INTP
9	Logistician	3.21	ISTJ
10	Executive	2.81	ESTJ
11	Debater	2.79	ENTP
12	Virtuoso	2.43	ISTP
13	Entrepreneur	2.16	ESTP
14	Architect	2.09	INTJ
15	Commander	1.26	ENTJ

We can gather the combined percentages for each of the personality abbreviations (I, E, N, S, F, T, J, P) by looping through the data and aggregating the percentages accordingly.

In [10]:

```
# Aggregate the percentages by the four traits
aggregated_data = defaultdict(float)

for index, row in df.iterrows():
    code = row['Code']
    percentage = row['Percentage']

    for char in code:
        aggregated_data[char] += percentage

# Create a DataFrame from the aggregated data
agg_df = pd.DataFrame(list(aggregated_data.items()), columns=['trait', 'Total Percentage'])
# Save the DataFrame to a CSV file for comparison analysis later on
agg_df.to_csv('agg_df.csv', index=False)

agg_df
```

Out[10]:

	trait	Total Percentage
0	I	55.73
1	N	49.30
2	F	79.30
3	P	58.57
4	E	44.26
5	S	50.69
6	J	41.42
7	T	20.69

Now that we have the combined percentages for each personality trait (I, E, N, S, F, T, J, P), we can further categorize these traits into their respective pairs: I/E (Introversion/Extraversion), S/N (Sensing/Intuition), F/T (Feeling/Thinking), and J/P (Judging/Perceiving) for comparison.

In [11]:

```
from collections import defaultdict

# Initialize the defaultdicts for each trait
aggregated_data_IE = defaultdict(float)
aggregated_data_NS = defaultdict(float)
aggregated_data_FT = defaultdict(float)
aggregated_data_JP = defaultdict(float)

# Aggregate the percentages for each trait
for index, row in df.iterrows():
    code = row['Code']
    percentage = row['Percentage']
```

```
    # Aggregate for 'I/E' trait
    if code[0] in ['I', 'E']:
        aggregated_data_IE[code[0]] += percentage

    # Aggregate for 'N/S' trait
    if code[1] in ['N', 'S']:
        aggregated_data_NS[code[1]] += percentage

    # Aggregate for 'F/T' trait
    if code[2] in ['F', 'T']:
        aggregated_data_FT[code[2]] += percentage

    # Aggregate for 'J/P' trait
    if code[3] in ['J', 'P']:
        aggregated_data_JP[code[3]] += percentage
```

Additionally, we can map these traits to their role groups and aggregate their percentages accordingly.

In [12]:
```
# Define the mappings of traits to role groups
role_mapping = {
    'NT': ['N', 'T'],
    'NF': ['N', 'F'],
    'SJ': ['S', 'J'],
    'SP': ['S', 'P']
}

# Initialize defaultdicts to aggregate percentages for each role group
aggregated_roles = defaultdict(float)

# Aggregate the percentages for each role group
for role, traits in role_mapping.items():
    for trait in traits:
        aggregated_roles[role] += aggregated_data[trait]

# Calculate the total percentage for all role groups
total_percentage_roles = sum(aggregated_roles.values())
```

2.2.3 Normalizing

Even though we might have retrieved the respective information for each pair, we need to normalize it. Normalization ensures that the percentages for each trait are relative and comparable. Without normalization, the raw percentages might be misleading because they are not scaled appropriately.

In [13]:
```
# Normalize the total percentages for each trait
total_percentage_IE = sum(aggregated_data_IE.values())
normalized_data_IE = {key: round((value / total_percentage_IE) * 100, 1) for key, value in aggregated_data_IE.i
agg_df_IE = pd.DataFrame(list(normalized_data_IE.items()), columns=['trait', 'Normalized Percentage'])

total_percentage_NS = sum(aggregated_data_NS.values())
normalized_data_NS = {key: round((value / total_percentage_NS) * 100, 1) for key, value in aggregated_data_NS.i
agg_df_NS = pd.DataFrame(list(normalized_data_NS.items()), columns=['trait', 'Normalized Percentage'])

total_percentage_FT = sum(aggregated_data_FT.values())
normalized_data_FT = {key: round((value / total_percentage_FT) * 100, 1) for key, value in aggregated_data_FT.i
agg_df_FT = pd.DataFrame(list(normalized_data_FT.items()), columns=['trait', 'Normalized Percentage'])

total_percentage_JP = sum(aggregated_data_JP.values())
normalized_data_JP = {key: round((value / total_percentage_JP) * 100, 1) for key, value in aggregated_data_JP.i
agg_df_JP = pd.DataFrame(list(normalized_data_JP.items()), columns=['trait', 'Normalized Percentage'])
```

In [14]:
```
# Normalize the percentages for each role group
agg_df_roles = pd.DataFrame(list(aggregated_roles.items()), columns=['Role Group', 'Total Percentage'])
agg_df_roles['Normalized Percentage'] = round((agg_df_roles['Total Percentage'] / total_percentage_roles) * 100
```

In [15]:
```
print(f"From a pool of 653,413 Singaporean respondents:")
# For I/E trait
print(f"For the 'Introverted' trait, {normalized_data_IE['I']}% of respondents exhibit this trait, while {norma

# For N/S trait
print(f"For the 'Intuitive' trait, {normalized_data_NS['N']}% of respondents possess this trait, while {normali

# For F/T trait
print(f"For the 'Feeling' trait, {normalized_data_FT['F']}% of respondents possess this trait, while {normalize

# For J/P trait
print(f"For the 'Judging' trait, {normalized_data_JP['J']}% of respondents possess this trait, while {normalize
```

```
From a pool of 653,413 Singaporean respondents:
For the 'Introverted' trait, 55.7% of respondents exhibit this trait, while 44.3% exhibit the 'Extraverted' trai
t. The difference is 11.4%.
For the 'Intuitive' trait, 49.3% of respondents possess this trait, while 50.7% exhibit the 'Sensing' trait. The
difference is 1.4%.
For the 'Feeling' trait, 79.3% of respondents possess this trait, while 20.7% exhibit the 'Thinking' trait. The
difference is 58.6%.
For the 'Judging' trait, 41.4% of respondents possess this trait, while 58.6% exhibit the 'Perceiving' trait. Th
e difference is 17.2%.
```

In [16]:
```python
# Print each role group with its respective percentage as a sentence
for index, row in agg_df_roles.iterrows():
    role_group_name = ""
    if row['Role Group'] == 'NT':
        role_group_name = "Analysts (xNTx)"
    elif row['Role Group'] == 'NF':
        role_group_name = "Diplomats (xNFx)"
    elif row['Role Group'] == 'SJ':
        role_group_name = "Sentinels (xSxJ)"
    elif row['Role Group'] == 'SP':
        role_group_name = "Explorers (xSxP)"
    print(f"For {role_group_name}, {row['Normalized Percentage']}% of respondents exhibit this role group.")
```

```
For Analysts (xNTx), 17.5% of respondents exhibit this role group.
For Diplomats (xNFx), 32.2% of respondents exhibit this role group.
For Sentinels (xSxJ), 23.0% of respondents exhibit this role group.
For Explorers (xSxP), 27.3% of respondents exhibit this role group.
```

In [17]:
```python
# Save each trait DataFrame to a CSV file
agg_df_IE.to_csv('trait_IE.csv', index=False)
agg_df_NS.to_csv('trait_NS.csv', index=False)
agg_df_FT.to_csv('trait_FT.csv', index=False)
agg_df_JP.to_csv('trait_JP.csv', index=False)
```

In [18]:
```python
agg_df_roles.to_csv('aggregated_role_data.csv', index=False)
```

2.3 Generating Features

2.3.1 Data Comparison

By extracting and generating features from the normalized aggregated data, we can conduct a comparison with global personality statistics. This comparative analysis allows us to explore the manifestation of various personality traits across diverse populations. As part of this analysis, we will be using NLTK's sentiment analyzer to generate sentiment measurements. For the sentiment analysis part, we used NLTK. It looks at the words in the comments to figure out if they sound happy, sad, or neutral. Since we didn't find much from Singapore, we focused on individual personality type groups on Reddit. NLTK's sentiment analysis assigns each comment a score based on its emotional tone. This score helps us understand whether the comment is positive, negative, or neutral. It's a neat way to see how people are feeling based on what they write.

In [19]:
```python
import pandas as pd

# Read the two CSV files into DataFrames
global_df = pd.read_csv('globaltraits.csv')
agg_df = pd.read_csv('agg_df.csv')

# Rename the columns to match the desired names
global_df.rename(columns={'Percentage': 'Global'}, inplace=True)
agg_df.rename(columns={'Total Percentage': 'Singapore'}, inplace=True)

# Find the common traits between the two DataFrames
common_traits = set(global_df['trait']).intersection(set(agg_df['trait']))

# Filter the DataFrames to include only the common traits
global_common = global_df[global_df['trait'].isin(common_traits)]
agg_common = agg_df[agg_df['trait'].isin(common_traits)]

# Merge the DataFrames on the 'trait' column to compare the percentages
merged_df = pd.merge(global_common, agg_common, left_on='trait', right_on='trait', suffixes=('_global', '_agg')

# Print the comparison DataFrame
print("Comparison of percentages for common traits:")
merged_df.head(8)
```

```
Comparison of percentages for common traits:
```

Out[19]:

	trait	Global	Singapore
0	E	50.0	44.26
1	I	50.0	55.73
2	S	45.0	50.69
3	N	55.0	49.30
4	F	54.5	79.30
5	T	45.4	20.69
6	J	48.8	41.42
7	P	51.2	58.57

In [20]:

```
# Read the aggregated role data CSV file into a DataFrame
aggregated_role_data_df = pd.read_csv('aggregated_role_data.csv')

# Read the global roles CSV file into a DataFrame
global_roles_df = pd.read_csv('globalroles.csv')

# Merge the two DataFrames based on the 'Role Group' column
rolemerged = pd.merge(aggregated_role_data_df, global_roles_df, on='Role Group', suffixes=('_aggregated', '_glol

# Drop the 'Total Percentage' column
rolemerged.drop(columns=['Total Percentage'], inplace=True)

# Rename the columns
rolemerged.rename(columns={'Normalized Percentage': 'Singapore', 'Percentage': 'Global'}, inplace=True)

# Print the comparison DataFrame
print("Comparison of percentages for common role groups:")
rolemerged.head()
```

```
Comparison of percentages for common role groups:
```

Out[20]:

	Role Group	Singapore	Global
0	NT	17.5	21.2
1	NF	32.2	40.7
2	SJ	23.0	23.0
3	SP	27.3	15.1

3 Case Study: Sentiment Analysis Using API

3.1 Authentication and Authorization for Reddit API

We thought it would be interesting to delve into sentiment analysis for each of the 16 unique Myers-Briggs Personality types using the Reddit API. To use the API, we needed to register an app on Reddit and obtain a client ID and a secret token. We will be using PRAW (Python Reddit API Wrapper) to easily interact with the API.

In [21]:

```
# Reddit API credentials
client_id = 'IFRBoI_KgBZs2OaMObs7SA'
client_secret = 'U_PKMReKN_ZWXfvAtRTuF2Xf0_GWUQ'
user_agent = 'WeatherSentimentAnalyzer by /u/Strict-Coyate-9298'

# MBTI types
mbti_types = ['INTJ', 'INTP', 'ENTJ', 'ENTP', 'INFJ', 'INFP', 'ENFJ', 'ENFP',
              'ISTJ', 'ISFJ', 'ESTJ', 'ESFJ', 'ISTP', 'ISFP', 'ESTP', 'ESFP']

# Initialize NLTK Sentiment Intensity Analyzer
sia = SentimentIntensityAnalyzer()

# Authenticate with Reddit API
reddit = praw.Reddit(
    client_id=client_id,
    client_secret=client_secret,
    user_agent=user_agent
)
```

3.2 Data cleaning and processing

To prepare the comments for sentiment analysis, we took the following steps:

- Cleaning the comments by removing irrelevant information like URLs, subreddit mentions, and non-alphanumeric characters.
- Breaking down the cleaned comments into individual words or tokens, a process called tokenization.

- Applying stemming to reduce each word to its base or root form.

3.2.1 Removing Irrelevant Information

We utilize the re.sub() function from Python's re module to clean the comments. This function allows us to remove unwanted elements such as URLs, subreddit mentions (like "r/subreddit"), and non-alphanumeric characters from the text data. This preprocessing step ensures that our text data is ready for sentiment analysis, where the presence of irrelevant information could skew the results.

In [22]:
```
# Function to remove irrelevant information from a comment
def remove_irrelevant_info(comment):
    return re.sub(r'http\S+|r/[^\s]+|u/[^\s]+|[^a-zA-Z0-9\s]', '', comment)

# http\S+: Removes URLs starting with "http".
# r/[^\s]+: Removes subreddit mentions in the format "r/subreddit".
# u/[^\s]+: Removes user mentions in the format "u/username".
# [^a-zA-Z0-9\s]: Removes non-alphanumeric characters, leaving only letters, digits, and spaces.
```

3.2.2 Tokenizing

We employs NLTK's word_tokenize() method to tokenize or break down a comment into individual words or tokens. This is a crucial step in natural language processing (NLP) tasks as it allows us to analyze text data at a more granular level. This process enables us to treat each word as a separate entity, facilitating further analysis.

In [23]:
```
# Function to tokenize a comment using NLTK
def tokenize_comment(comment):
    tokenized_comment = word_tokenize(comment)
    return tokenized_comment
```

3.2.3 Stemming

We use stemming to standardize words in our subreddit comments. This simplifies the text data by reducing words to their root forms, such as "running" and "runner" becoming "run". This standardization facilitates analysis and comparison of the text data.

In [24]:
```
# Function to apply stemming using NLTK's PorterStemmer
def stem_words(words):
    ps = PorterStemmer()
    return [ps.stem(word) for word in words]
```

3.3 Generating features

3.3.1 Sentiment Analysis

We used NLTK's Sentiment Intensity Analyzer to evaluate the sentiment of the cleaned and processed comments. Each comment was assigned a sentiment score, which was then recorded along with its MBTI type and the original comment. For this analysis, we will be searching for 10 comments in each MBTI type subreddit. This is to ensure that we have a diverse and representative sample of comments for each personality type. By analyzing these samples, we can draw more accurate conclusions about the overall sentiment and common traits of each MBTI type.

In [25]:
```
# Function to search for comments in each MBTI type subreddit
def search_comments(subreddit_name):
    try:
        subreddit = reddit.subreddit(subreddit_name)
        comment_count = 0
        for submission in subreddit.new(limit=10):
            submission.comments.replace_more(limit=None)
            for comment in submission.comments.list():
                if comment_count >= 10:
                    break
                comment_count += 1

    except Exception as e:
        print(f"Error fetching comments for subreddit r/{subreddit_name}: {e}")
```

In [26]:
```
# Function to perform sentiment analysis using NLTK's SentimentIntensityAnalyzer
def analyze_sentiment(comment):
    scores = sia.polarity_scores(comment)
    return scores['compound']

# Initialize DataFrame to store sentiment analysis results
sentiment_df = pd.DataFrame(columns=['MBTI Type', 'Comment', 'Sentiment Score'])

# Function to search for comments in each MBTI type subreddit
def search_comments(subreddit_name):
    try:
        subreddit = reddit.subreddit(subreddit_name)
        comment_count = 0
```

```
        for submission in subreddit.new(limit=10):
            submission.comments.replace_more(limit=None)
            for comment in submission.comments.list():
                if comment_count >= 10:
                    break
                comment_body = comment.body
                cleaned_comment = remove_irrelevant_info(comment_body)
                tokenized_comment = tokenize_comment(cleaned_comment)
                stemmed_comment = stem_words(tokenized_comment)
                sentiment_score = analyze_sentiment(' '.join(stemmed_comment))
                sentiment_df.loc[len(sentiment_df)] = [subreddit_name, comment_body, sentiment_score]
                comment_count += 1
    except Exception as e:
        print(f"Error fetching comments for subreddit r/{subreddit_name}: {e}")

# Search for comments and perform sentiment analysis for each MBTI type subreddit
for mbti_type in mbti_types:
    search_comments(mbti_type)
```

In [27]:
```
# Group sentiment DataFrame by MBTI type and calculate mean sentiment score
average_sentiment_per_mbti = sentiment_df.groupby('MBTI Type')['Sentiment Score'].mean()

# Print the average sentiment score for each MBTI type
print("Average Sentiment Score for Each MBTI Type:")
print(average_sentiment_per_mbti)
```

```
Average Sentiment Score for Each MBTI Type:
MBTI Type
ENFJ    0.64994
ENFP    0.21091
ENTJ    0.66561
ENTP   -0.01731
ESFJ    0.31421
ESFP    0.32173
ESTJ    0.38945
ESTP    0.20498
INFJ    0.02871
INFP    0.08156
INTJ    0.15460
INTP   -0.13395
ISFJ    0.63851
ISFP    0.33425
ISTJ   -0.11844
ISTP    0.26904
Name: Sentiment Score, dtype: float64
```

3.3.2 Finding Common Words Among Most and Least Sentiment MBTI Types

We discovered that ESTP has the lowest sentiment score, while ESFP has the highest sentiment score! With our data now cleaned, we can identify the most frequent words in all the comments for each personality type. This will help us further analyze and understand the common traits shared by individuals within each MBTI type. By examining these frequent words, we can gain deeper insights into the unique characteristics and perspectives of each personality group.

In [28]:
```
# Define the list of stopwords
stop_words = set(stopwords.words('english'))

# Filter the DataFrame for ESFP comments
esfp_comments = sentiment_df[sentiment_df['MBTI Type'] == 'ESFP']['Comment']

# Initialize a list to store all words for ESFP
all_esfp_words = []

# Process each ESFP comment to extract words
for comment in esfp_comments:
    cleaned_comment = remove_irrelevant_info(comment)
    tokenized_comment = tokenize_comment(cleaned_comment)
    stemmed_comment = stem_words(tokenized_comment)
    all_esfp_words.extend(stemmed_comment)

# Remove stopwords for ESFP
filtered_esfp_words = [word for word in all_esfp_words if word not in stop_words]
```

In [29]:
```
# Filter the DataFrame for ESTP comments
estp_comments = sentiment_df[sentiment_df['MBTI Type'] == 'ESTP']['Comment']

# Initialize a list to store all words for ESTP
all_estp_words = []

# Process each ESTP comment to extract words
for comment in estp_comments:
    cleaned_comment = remove_irrelevant_info(comment)
    tokenized_comment = tokenize_comment(cleaned_comment)
```

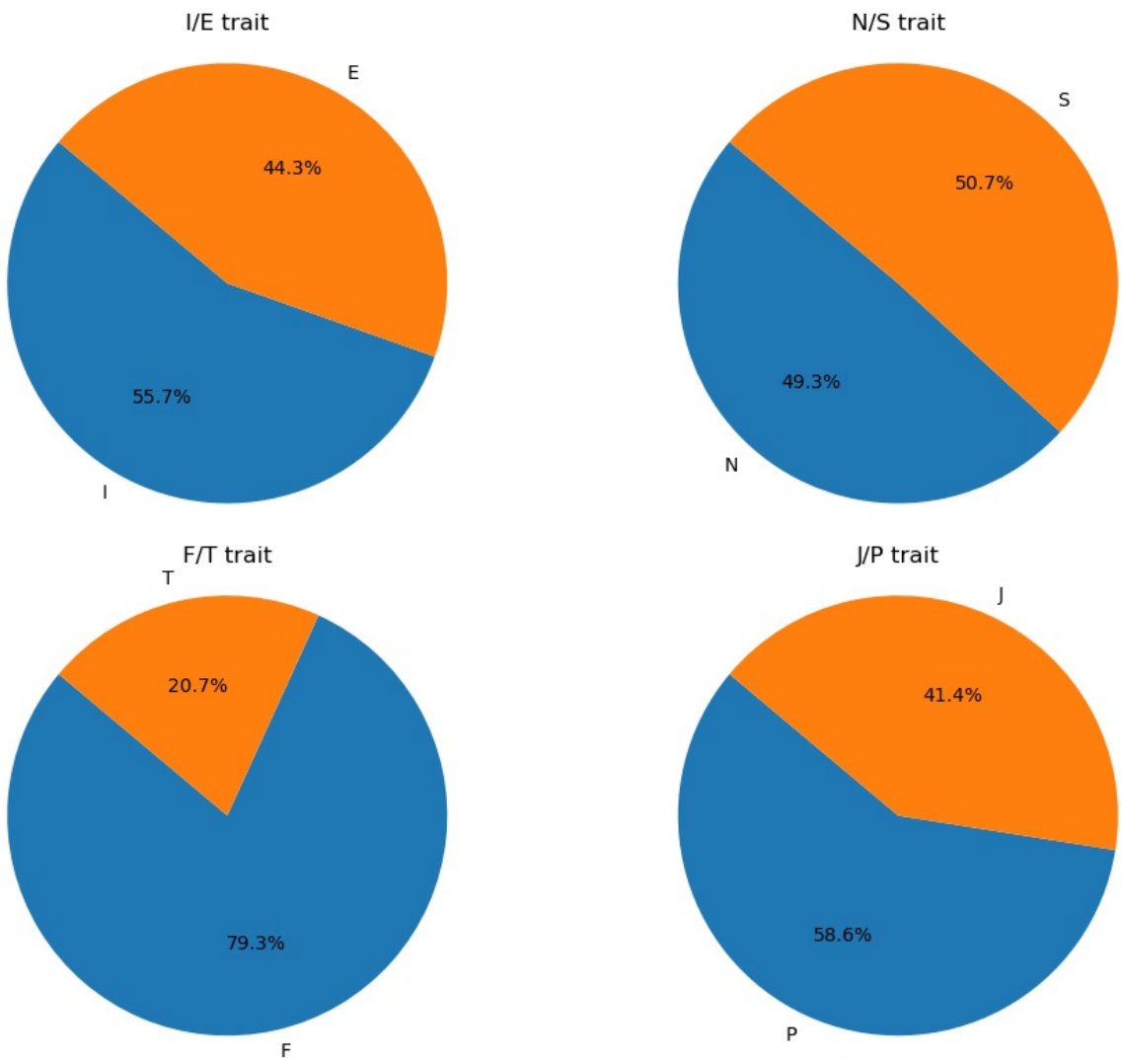
```
    stemmed_comment = stem_words(tokenized_comment)
    all_estp_words.extend(stemmed_comment)

# Remove stopwords for ESTP
filtered_estp_words = [word for word in all_estp_words if word not in stop_words]
```

4 Exploratory data analysis

4.1 Comparative Analysis of Personality Traits

We can visualize the data to observe the percentage distribution of each trait and make interpretations using the normalized aggregated dataset generated earlier (2.3.1).

In [30]:
```
# Read the CSV files into DataFrames
df_IE = pd.read_csv('trait_IE.csv')
df_NS = pd.read_csv('trait_NS.csv')
df_FT = pd.read_csv('trait_FT.csv')
df_JP = pd.read_csv('trait_JP.csv')

# Function to plot pie chart for a trait
def plot_pie_chart(ax, df, trait_name):
    ax.pie(df['Normalized Percentage'], labels=df['trait'], autopct='%1.1f%%', startangle=140)
    ax.set_title(f'{trait_name} trait')
    ax.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle

# Create a 2x2 grid of subplots
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 8))

# Plot pie charts for each trait
plot_pie_chart(axes[0, 0], df_IE, 'I/E')
plot_pie_chart(axes[0, 1], df_NS, 'N/S')
plot_pie_chart(axes[1, 0], df_FT, 'F/T')
plot_pie_chart(axes[1, 1], df_JP, 'J/P')

# Adjust layout
plt.tight_layout()

# Show the plot
plt.show()
```

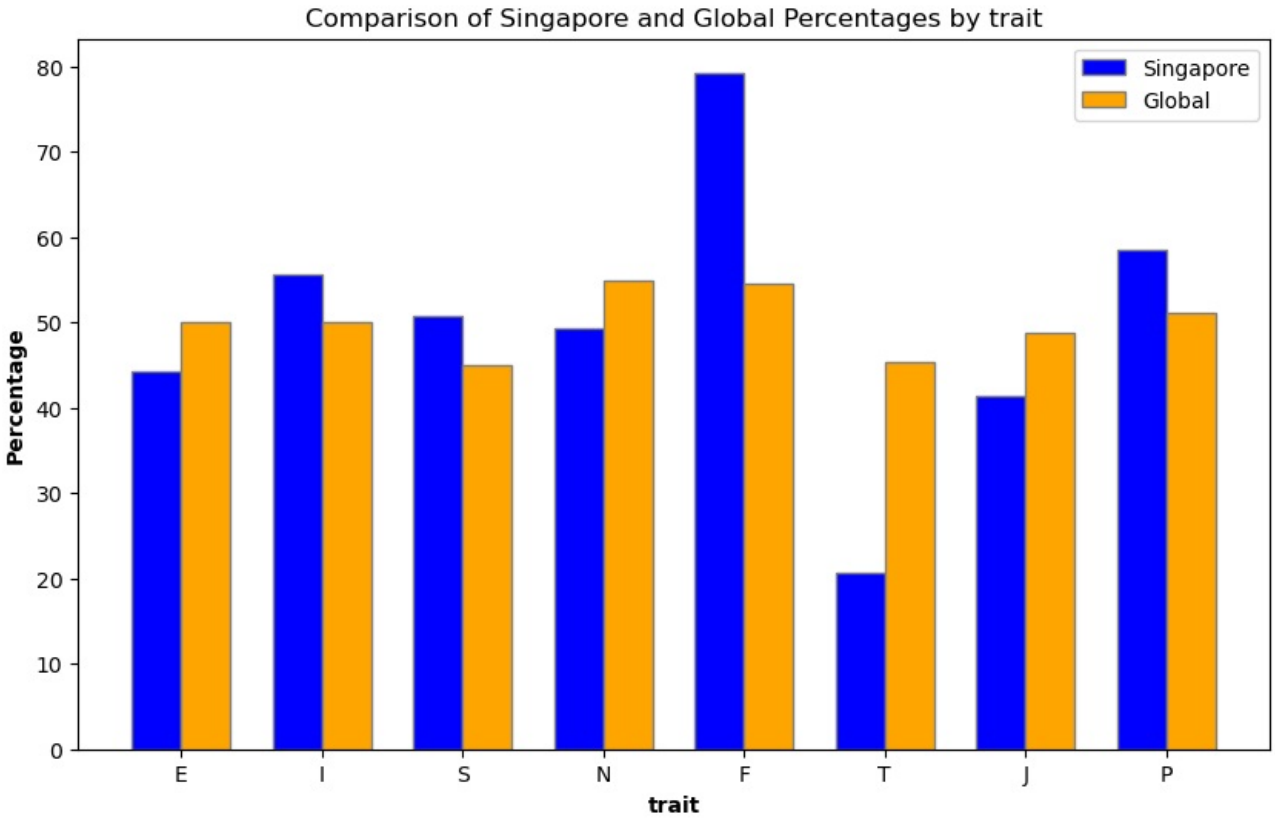


From the generated plots, we can observe the dominant personality traits among Singaporeans:

- Introversion: Most Singaporeans are introverted, likely due to the structured nature of Singapore's education and work environments, which emphasize individual achievement and deep thinking.
- Sensing and Intition: The almost equal percentage of Sensing and Intuition in Singapore reflects a balanced approach to both practical, detail-oriented thinking and abstract, innovative thinking. This balance is nurtured by the education system, economic needs, cultural influences, workplace diversity, and government policies.
- Feeling: Singaporean culture values harmony and caring for others' feelings, matching the Feeling preference in the MBTI. This preference prioritizes empathy and the impact of decisions on others.
- Perceiving: Singaporeans tend to score higher in Perceiving rather than Judging due to cultural values of adaptability, flexibility, and resilience. The education and career environments also emphasize innovation and creativity, which align with the Perceiving preference.

We can also analyze the distribution of traits between Singapore and global's dataset:

In [31]:
```
# Define traits and corresponding percentages for Singapore and Global
traits = merged_df['trait']
singapore_percentages = merged_df['Singapore']
global_percentages = merged_df['Global']

# Set the width of the bars
bar_width = 0.35

# Set the position of each bar on the x-axis
r1 = range(len(traits))
r2 = [x + bar_width for x in r1]

# Plot the data
plt.figure(figsize=(10, 6))
plt.bar(r1, singapore_percentages, color='blue', width=bar_width, edgecolor='grey', label='Singapore')
plt.bar(r2, global_percentages, color='orange', width=bar_width, edgecolor='grey', label='Global')

# Add xticks on the middle of the group bars
plt.xlabel('trait', fontweight='bold')
plt.ylabel('Percentage', fontweight='bold')
plt.xticks([r + bar_width/2 for r in range(len(traits))], traits)

# Add a legend
plt.legend()

# Show plot
plt.title('Comparison of Singapore and Global Percentages by trait')
plt.show()
```



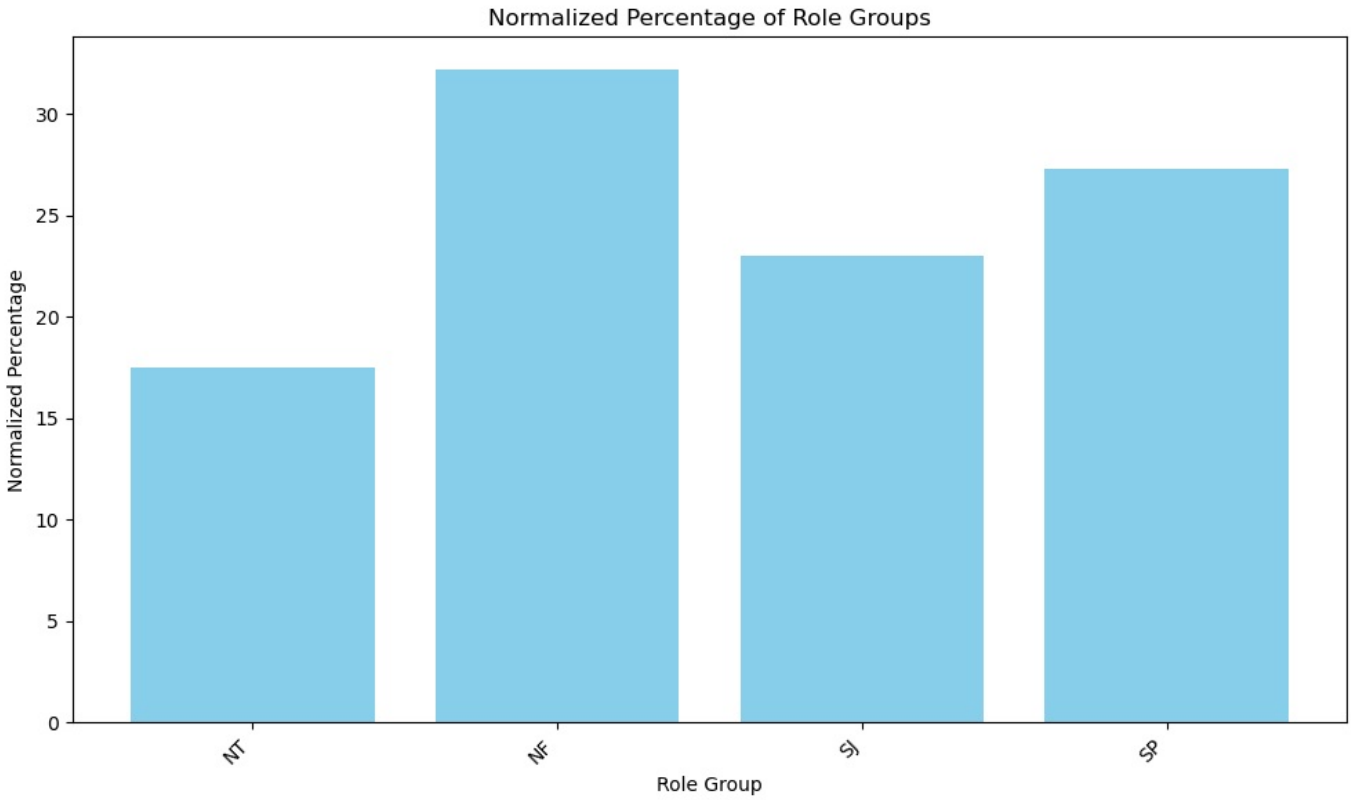
It is clear that Singaporeans score a higher in Introversion (I), Sensing (S), Feeling (F), and Perceiving (P) compared to global averages which can be attributed to several cultural, societal, and environmental factors.

- Introversion (I): Singaporeans tend to be more introverted due to a structured education system and cultural norms valuing reserved behavior. Globally, there's often a balance or slight preference for Extraversion, emphasizing sociability.
- Sensing (S): Singaporeans favor Sensing, focusing on practicality and detail. Globally, there's a mix, with some cultures leaning towards Intuition, which emphasizes abstract thinking and innovation.
- Feeling (F): Singaporeans generally score higher in Feeling, valuing empathy and harmony. The global average may be more balanced or slightly higher in Thinking, which prioritizes logical analysis.
- Perceiving (P): Singaporeans often score higher in Perceiving, reflecting adaptability and flexibility. Globally, there may be a more even split or a slight preference for Judging, which focuses on structure and planning.

4.2 Comparative Analysis of Personality roles

To deepen our analysis, we can explore the personality traits catogorise in different role groups. We can visualize the data to identify patterns and trends within each category using the normalized aggregated dataset generated earlier.

In [32]:
```
# Data for the bar chart
role_groups = agg_df_roles['Role Group']
normalized_percentages = agg_df_roles['Normalized Percentage']

# Create bar chart
plt.figure(figsize=(10, 6))
plt.bar(role_groups, normalized_percentages, color='skyblue')
plt.title('Normalized Percentage of Role Groups')
plt.xlabel('Role Group')
plt.ylabel('Normalized Percentage')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

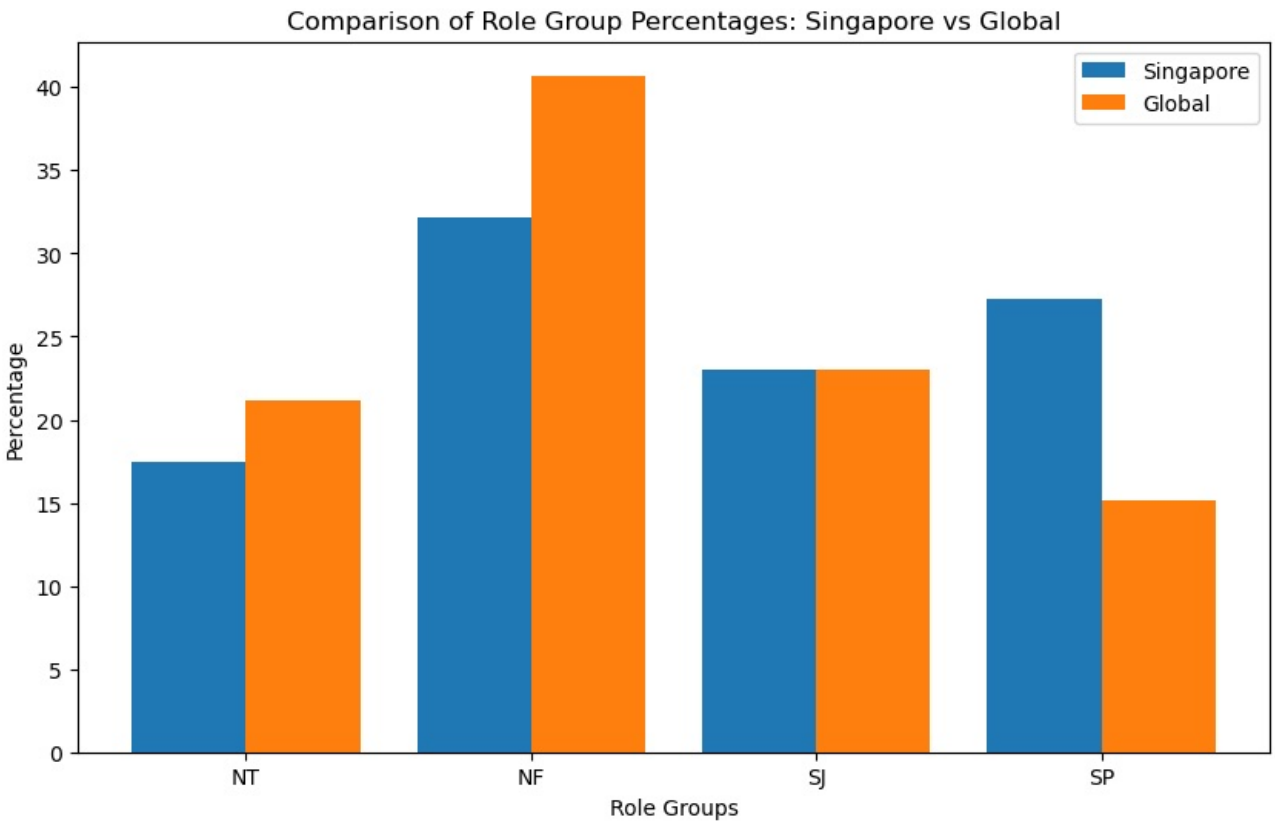


We observed that the majority of Singaporeans are Diplomats, sharing the Intuitive and Feeling traits (NF), while Analysts, who share the Intuitive and Thinking traits (NT), are the least common. The lower prevalence of NT types among Singaporeans could be due to cultural values that emphasize social harmony, empathy, and teamwork, which align with NF traits. The education system and workplaces focus on emotional intelligence and collaboration, supporting NF characteristics. Consequently, NT traits like analytical thinking and strategic planning are less prominent in Singaporean society.

We can also analyze the distribution of roles between Singapore and global's dataset:

In [33]:
```
# Plotting the data
fig, ax = plt.subplots(figsize=(10, 6))

# Set the positions and width for the bars
```

```
positions = range(len(rolemerged))
width = 0.4

# Plot bars
ax.bar(positions, rolemerged['Singapore'], width=width, label='Singapore', align='center')
ax.bar([p + width for p in positions], rolemerged['Global'], width=width, label='Global', align='center')

# Set the labels and title
ax.set_xlabel('Role Groups')
ax.set_ylabel('Percentage')
ax.set_title('Comparison of Role Group Percentages: Singapore vs Global')
ax.set_xticks([p + width/2 for p in positions])
ax.set_xticklabels(rolemerged['Role Group'])
ax.legend()

# Show the plot
plt.show()
```



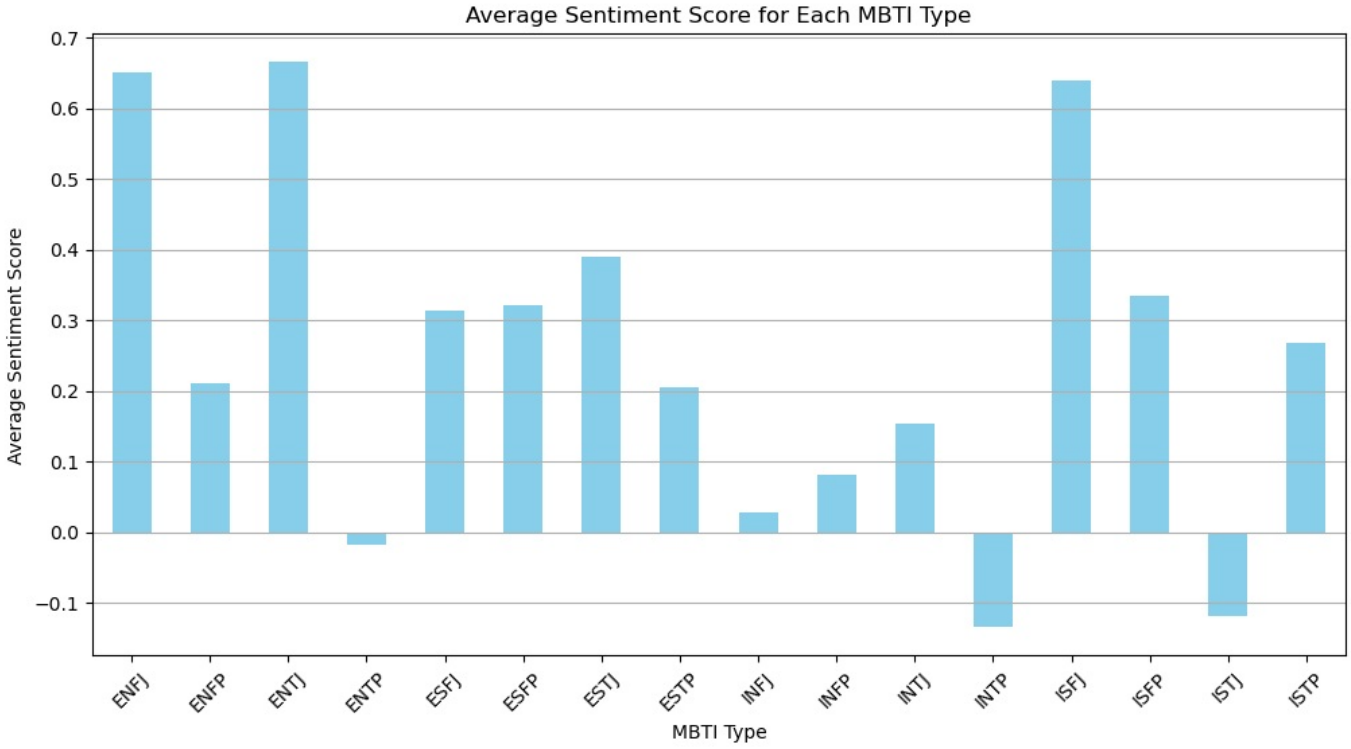
Singapore has a higher proportion of Explorer types compared to the global average, which may be due to its dynamic and rapidly changing environment. The country's emphasis on innovation, adaptability, and exploration aligns with the traits of Explorers. This role values creativity, spontaneity, and flexibility, which are essential in Singapore's fast-paced and evolving landscape. Additionally, the diverse and multicultural environment encourages individuals to embrace new experiences and ideas, further contributing to the higher presence of Explorers in Singapore. Similarly, as with the previous data, the proportion of Analyst types among Singaporeans remains the lowest compared to the global average.

4.3 Top Words/Word Cloud

We conducted a sentiment analysis for each of the 16 Myers-Briggs Personality types by collecting comments from specific subreddits. After cleaning the data by removing irrelevant information, tokenizing, and stemming the comments, we used NLTK to compute the average sentiment score for each personality type.

We then visualized the data by plotting the sentiment values for eachpersonality type in a bar charts to find larger trends. We discovered that the personality type with the lowest sentiment score is ESTP and the personality type with the highest sentiment score is ESFP.

In [34]:
```
# Plot the average sentiment scores as a bar graph
plt.figure(figsize=(12, 6))
average_sentiment_per_mbti.plot(kind='bar', color='skyblue')
plt.title('Average Sentiment Score for Each MBTI Type')
plt.xlabel('MBTI Type')
plt.ylabel('Average Sentiment Score')
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.show()
```

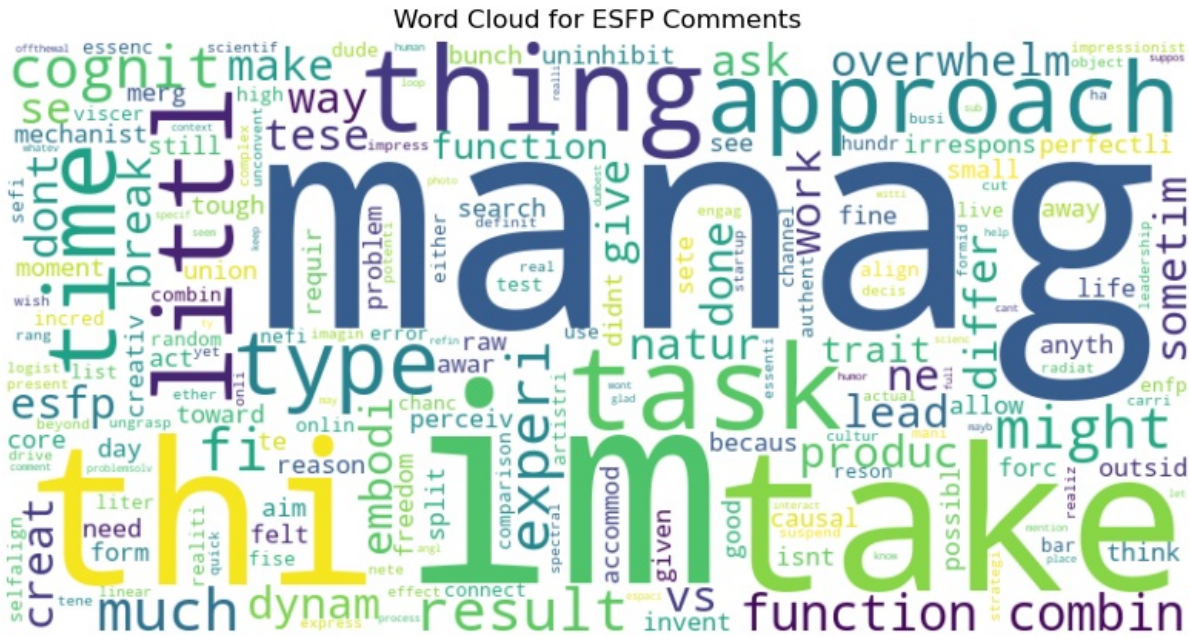


We looked at the words used by each personality type by creating word clouds. We found the 10 most common words for the most positive and least positive personality types. "Think" was the most-used word for both, showing that these groups like to reflect and analyze a lot. In the ESFP personality type, the most frequent word was "philosophy," indicating a strong interest in deep and thoughtful topics, which might contribute to higher sentiment.

For the ESTP personality type, common words included "don't" and "lol." The word "don't" can contribute to a lower sentiment because it's often used in negative or critical contexts. On the other hand, "lol" might indicate a casual, sometimes dismissive tone, which could be perceived as less serious or genuine, potentially contributing to a lower overall sentiment. This means that, despite different overall sentiments, the main topics in these groups reflect their distinct communication styles and focus areas.

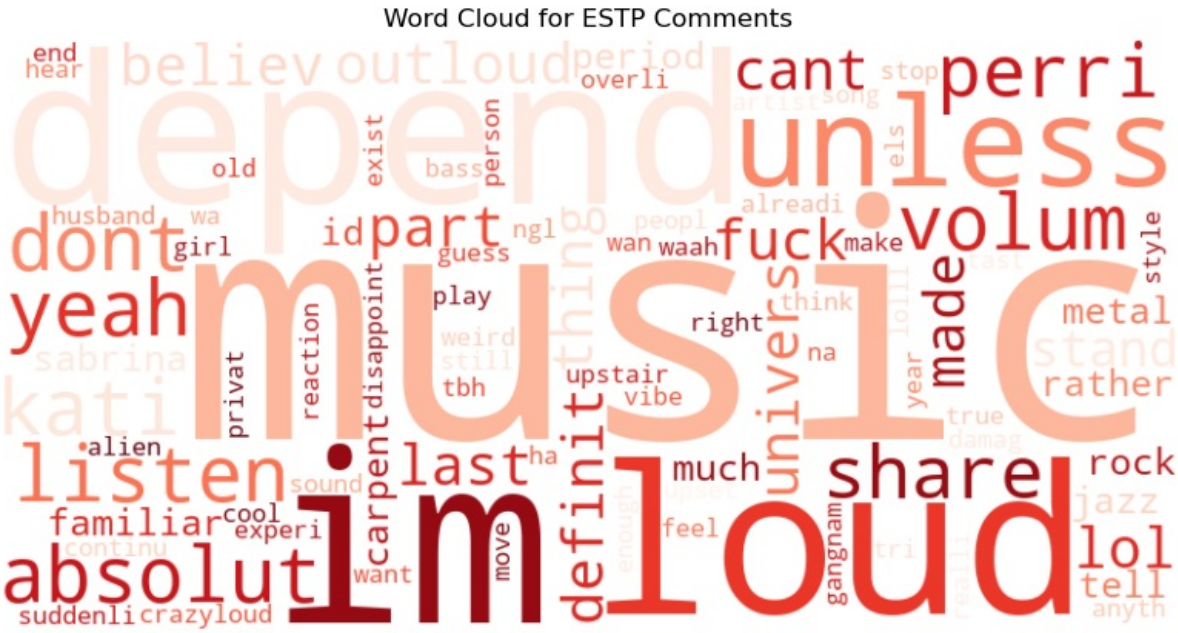
Below are the word clouds for ESFP and ESTP personalities:

In [35]:
```
# Generate a word cloud for ESFP comments
esfp_wordcloud = WordCloud(width=800, height=400, background_color='white').generate(' '.join(filtered_esfp_wor

# Display the word cloud for ENFP comments using matplotlib
plt.figure(figsize=(10, 5))
plt.imshow(esfp_wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud for ESFP Comments')
plt.show()
```


Word Cloud for ESFP Comments
manag
think
take
thing
approach
time
little
task
type
experi
function
combin
result
overwhelm
ask
make
way
tese
cognit
essenc
merg
mechanist
still
break
dont
tough
union
combin
random
act
toward
esfp
core
day
fi
reason
aim
felt
need
form
much
dynam
creat
selfalign
creativ
search
give
done
work
fine
differ
sometim
might
lead
produc
becaus
trait
natur
perceiv
chanc
onlin
embodi
freedom
split
comparison
accommod
given
vs
invent
connect
effect
possibl
causal
isnt
good
think
bar
outsid
forc
reson
anyth
life
away
small
irrespons
perfectli
hundr
busi
see
function
impress
problem
require
either
didnt
sete
raw
error
use
neti
awar
ne
allow
enfp
carri
realiz
strategi
spectral
artistri
bunch
uninhibit
dude
scientif
high
unconvent
complex
context
seen
keep
potenti
definit
real
test
startup
channel
authent
essenti
humor
actual
cultur
mani
nayb
wont
glad
onli
ether
yet
imagin
refin
ungrasp
beyond
drive
comment
problemsolv
liter
realiti
quick
fise
nete
linear
tene
express
process
espacio
spectral
interact
suspend
know
mention
place
let
nayb
ha
sub
cut
help
formid
leadership
cant
radiat
carri
impressionist
object
suppos
busi
hundr
decis
align
live
formid

meaningful concepts, which can be uplifting and engaging.

In [36]:
```
# Generate a word cloud for ESTP comments
estp_wordcloud = WordCloud(width=800, height=400, background_color='white', colormap='Reds').generate(' '.join(

# Display the word cloud for ESTP comments using matplotlib
plt.figure(figsize=(10, 5))
plt.imshow(estp_wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud for ESTP Comments')
plt.show()
```



5 Summary

5.1 Conclusion

In conclusion, the personality traits of Singaporeans show a distinct pattern influenced by cultural, educational, and societal factors. Singaporeans are generally more Introverted, Sensing, Feeling, and Perceiving compared to global averages. This reflects a preference for individual achievement, practical and detail-oriented thinking, empathy, adaptability, and flexibility. The majority of Singaporeans fall into the Diplomat category, emphasizing empathy and cooperation, while Explorers are more prevalent due to the dynamic and innovative environment. These insights highlight how Singapore's structured education system and cultural values shape its unique personality landscape.

6 References and Resources

5.1 References

1. Kaggle Dataset:
   - die9origephit. (n.d.). MBTI Personality Type Test Complete Dataset. Kaggle. Retrieved from https://www.kaggle.com/datasets/die9origephit/mbti-personality-type-test-complete-dataset
2. 16Personalities Country Profiles:
   - 16Personalities. (n.d.). World: 16Personalities Country Profiles. Retrieved from https://www.16personalities.com/country-profiles/global/world#global
3. Medium Article:
   - Garage Gouch. (n.d.). MBTI Data Analysis. Medium. Retrieved from https://medium.com/@garagegouch/mbti-data-analysis-1bbad3b40cc9
4. Kaggle Code (MBTI Personality Predictor):
   - Rajshreev. (n.d.). MBTI Personality Predictor Using Machine Learning. Kaggle. Retrieved from https://www.kaggle.com/code/rajshreev/mbti-personality-predictor-using-machine-learning#Pre-Processing-Stage
5. Kaggle Code (MBTI x Birthday Analysis):
   - Lawson, M. (n.d.). MBTI x Birthday Analysis. Kaggle. Retrieved from https://www.kaggle.com/code/michellelawson/mbti-x-birthday-analysis
6. LinkedIn Advice:

- LinkedIn. (n.d.). What are the most effective ways to collect data from Reddit? Retrieved from https://www.linkedin.com/advice/1/what-most-effective-ways-collect-data-from-reddit-ybgtf

7. Simply Psychology Article:

- McLeod, S. (n.d.). The Myers-Briggs Type Indicator. Simply Psychology. Retrieved from https://www.simplypsychology.org/the-myers-briggs-type-indicator.html

5.2 Resources

Web Scraping for MBTI Personality trait Analysis:

1. **Requests Documentation**:

- Requests: HTTP for Humans. Retrieved from https://docs.python-requests.org/en/latest/

2. **BeautifulSoup Documentation**:

- BeautifulSoup Documentation. Retrieved from https://www.crummy.com/software/BeautifulSoup/bs4/doc/

Data cleaning and processing and generating Features:

3. **pandas.read_csv**:

- pandas.read_csv — pandas 1.3.0 documentation. Retrieved from https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html

4. **pandas.DataFrame.head**:

- pandas.DataFrame.head — pandas 1.3.0 documentation. Retrieved from https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.head.html

5. **pandas.DataFrame.iterrows**:

- pandas.DataFrame.iterrows — pandas 1.3.0 documentation. Retrieved from https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.iterrows.html

6. **pandas.DataFrame.to_csv**:

- pandas.DataFrame.to_csv — pandas 1.3.0 documentation. Retrieved from https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_csv.html

7. **collections.defaultdict**:

- collections.defaultdict — The Python Standard Library. Retrieved from https://docs.python.org/3/library/collections.html#collections.defaultdict

8. **Python built-in sum() function**:

- Built-in Functions — Python 3.10.1 documentation. Retrieved from https://docs.python.org/3/library/functions.html#sum

9. **pandas.DataFrame**:

- pandas.DataFrame — pandas 1.3.0 documentation. Retrieved from https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.html

10. **pandas.DataFrame.rename**:

- pandas.DataFrame.rename — pandas 1.3.0 documentation. Retrieved from https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.rename.html

11. **pandas.DataFrame.merge**:

- pandas.DataFrame.merge — pandas 1.3.0 documentation. Retrieved from https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.merge.html

12. **pandas.DataFrame.drop**:

- pandas.DataFrame.drop — pandas 1.3.0 documentation. Retrieved from https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.drop.html

Case Study: Sentiment Analysis Using API:

13. **Regular Expressions (re.sub)**:

- Python's re.sub function: www.docs.python.org/3/library/re.html#re.sub

14. **NLTK (Natural Language Toolkit**:

- NLTK Tokenization Documentation: www.nltk.org/api/nltk.tokenize.html#nltk.tokenize.word_tokenize
- NLTK Stemming Documentation: www.nltk.org/api/nltk.stem.html#nltk.stem.porter.PorterStemmer

15. **PRAW (Python Reddit API Wrapper)**:

- PRAW Documentation: www.praw.readthedocs.io/en/latest/

NLTK's SentimentIntensityAnalyzer (sia):

- NLTK Sentiment Analysis Documentation: www.nltk.org/api/nltk.sentiment.html#module-nltk.sentiment.vader

pandas DataFrame Operations:

- pandas DataFrame Documentation: www.pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.html

NLTK's stopwords:

- NLTK Stopwords Documentation: www.nltk.org/book/ch02.html#stopwords-indexhtml#stopwords-index)

Exploratory data analysis:

16. **Matplotlib Documentation**:

- Matplotlib: Visualization with Python. Retrieved from https://matplotlib.org/stable/contents.html

17. **pandas.DataFrame.plot**:

- pandas.DataFrame.plot — pandas 1.3.0 documentation. Retrieved from https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.plot.html

18. **WordCloud Visualization**:

- WordCloud Documentation. Retrieved from https://github.com/amueller/word_cloud

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js